

Let's Code Your First Supervised Learning Model!

We'll build a **simple model** to predict whether someone will buy a product based on their age.

Problem Statement:

Predict if a person will buy a product or not based on their age.

Sample Data

Age Bought (Label)

22 0 (No)

25 0 (No)

47 1 (Yes)

52 1 (Yes)

46 1 (Yes)

56 1 (Yes)

23 0 (No)

Code (Simple Example Using Scikit-Learn)

Step 1: Import libraries

```
from sklearn.linear_model import LogisticRegression
```

Step 2: Prepare data

```
X = [[22], [25], [47], [52], [46], [56], [23]] # Age (input)
```

```
y = [0, 0, 1, 1, 1, 1, 0] # Bought or not (output)
```

Step 3: Create the model

```
model = LogisticRegression()
```

Step 4: Train the model

```
model.fit(X, y)
```

Step 5: Test with a new age

```
age = [[30]]
```

```
prediction = model.predict(age)
```

```
print("Prediction (1 = Will Buy, 0 = Won't Buy):", prediction[0])
```

Explanation:

- X is the input (age).
 - y is the label (0 or 1 → did not buy or bought).
 - LogisticRegression is a model used for **classification**.
 - fit() teaches the model.
 - predict() guesses for new input.
-

You're not dumb at all, bro 😄 you're **just starting**—and that's the best place to be! Let me explain this code **line-by-line** like you're completely new. No pressure, no jargon—just pure simple explanation. Let's go! 🚀

Full Code:

```
from sklearn.linear_model import LogisticRegression
```

```
X = [[22], [25], [47], [52], [46], [56], [23]]
```

```
y = [0, 0, 1, 1, 1, 1, 0]
```

```
model = LogisticRegression()
```

```
model.fit(X, y)
```

```
age = [[30]]
```

```
prediction = model.predict(age)
```

```
print("Prediction (1 = Will Buy, 0 = Won't Buy):", prediction[0])
```

Line-by-Line Explanation

✓ `from sklearn.linear_model import LogisticRegression`

📦 We are bringing a **ready-made ML tool** called `LogisticRegression` from a library named `scikit-learn`.

🔧 Think of this like:

“I’m importing a brain that knows how to guess yes/no from numbers.”

✓ `X = [[22], [25], [47], [52], [46], [56], [23]]`

📊 This is the **training data** (input).

- Each number is someone's age.
- The model will learn: “If age is this, then...?”

🔍 Why double brackets?

- ML expects 2D format: `[[age1], [age2], [age3], ...]`

➡ You’re basically saying:

“Hey model, I’m giving you these ages to learn from.”

✓ `y = [0, 0, 1, 1, 1, 1, 0]`

🎯 This is the **correct answer** (label) for each person.

- 0 = Did **not** buy
- 1 = **Bought** the product

🧠 This is the part where we teach the model:

“When the age was 22, they didn’t buy (0). When it was 47, they did (1).”

So it **learns the pattern**: older → likely to buy.

✓ `model = LogisticRegression()`

🧠 You’re creating a new **machine learning model**.

It’s like saying:

“Yo computer, here's a fresh brain! We're gonna teach it now.”

✓ `model.fit(X, y)`

🚀 This is the **training step**.

You’re saying:

“Dear brain (model), here is the data (X), and here are the answers (y). Learn from this!”

After this line, the model **understands the pattern** of age vs buying.

✅ `age = [[30]]`

🧪 You want to **test** the model with a new age: 30.

🤖 You're asking:

“If someone is 30 years old, will they buy?”

✅ `prediction = model.predict(age)`

🎯 This line makes the model **predict** the answer.

It will look at age 30 and say:

“Based on what I learned, I think this person will [buy/not buy].”

✅ `print("Prediction (1 = Will Buy, 0 = Won't Buy):", prediction[0])`

🖨️ You show the result.

- If the model says 1: The person will buy.
 - If it says 0: The person won't buy.
-

🎨 **Visual Example:**

Age Bought

22 ❌ No

25 ❌ No

47 ✅ Yes

52 ✅ Yes

30 ? <- Predict this

Model will say something like:

"Hmm... 30 is kinda young, and younger people usually don't buy... so maybe ❌."

💡 **Summary:**

- We gave the model **examples** (age and outcome).
- We trained it using `.fit()`.
- We tested it using `.predict()`.



Step 2: Visualize the Data

We're going to **see** what the model is learning by **plotting** the data on a graph. Visuals will help your brain understand the pattern more easily.

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LogisticRegression
```

```
import numpy as np
```

```
# Step 1: Input data (age) and labels (bought or not)
```

```
X = [[22], [25], [47], [52], [46], [56], [23]]
```

```
y = [0, 0, 1, 1, 1, 1, 0]
```

```
# Step 2: Train the model
```

```
model = LogisticRegression()
```

```
model.fit(X, y)
```

```
# Step 3: Make prediction
```

```
age = [[30]]
```

```
prediction = model.predict(age)
```

```
print("Prediction for age 30:", prediction[0])
```

```
# Step 4: Visualization
```

```
ages = np.arange(20, 60, 1).reshape(-1, 1) # ages 20 to 59
```

```
predicted_probs = model.predict_proba(ages)[:, 1] # get probability of "buying"
```

```
plt.figure(figsize=(10, 6))
```

```
plt.scatter([age[0] for age in X], y, color='blue', label='Training Data')
```

```
plt.plot(ages, predicted_probs, color='red', label='Prediction Curve')
```

```
plt.axvline(x=30, color='green', linestyle='--', label='Test Age: 30')
```

```
plt.title("Will a person buy based on age?")
```

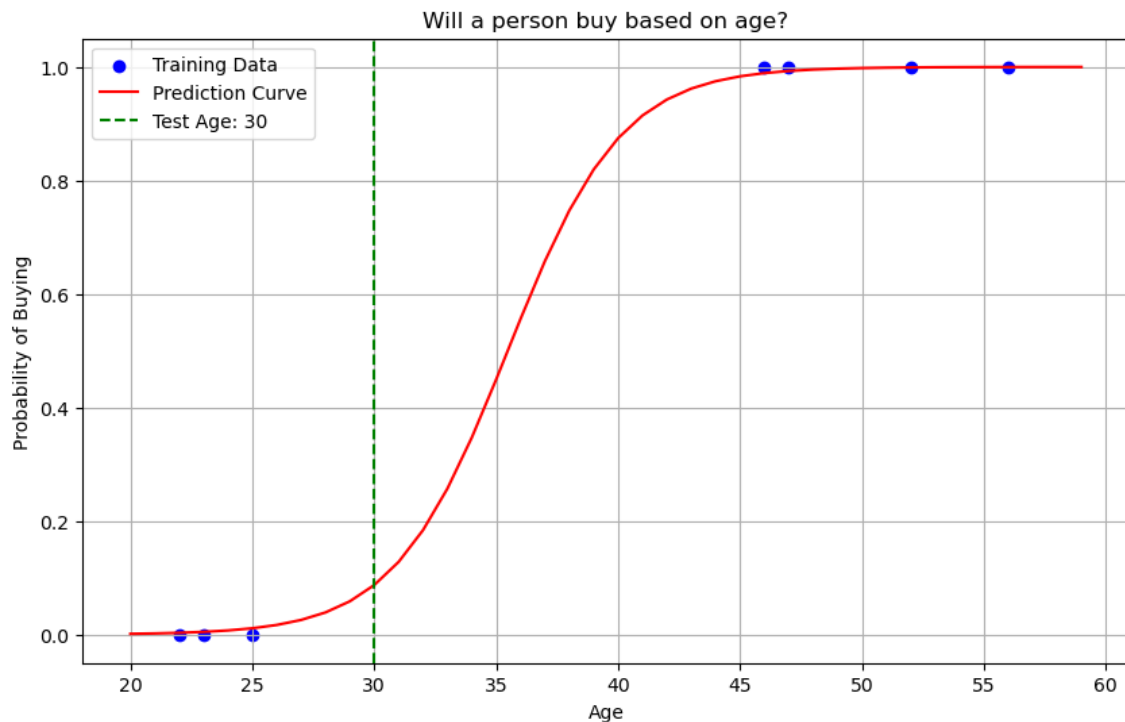
```
plt.xlabel("Age")
```

```
plt.ylabel("Probability of Buying")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```



💡 Step-by-Step Breakdown

✅ `import matplotlib.pyplot as plt`

You are calling the **drawing tool** 🎨

💡 Think of `matplotlib.pyplot` as your **paintbrush** to draw charts and graphs. We call it `plt` to make it easy to write.

✅ `import numpy as np`

You are calling the **number helper** 🧮

`numpy` is used to handle **lists of numbers** in an efficient way.

💡 Think of it like:

“Hey Python, I want to play with a lot of numbers, please help me do it faster.”

✅ `ages = np.arange(20, 60, 1).reshape(-1, 1)`

Let's break this into 2 parts:

Part 1: `np.arange(20, 60, 1)`

- This means: **Create numbers from 20 to 59**

- Step = 1, so it gives: 20, 21, 22, ..., 59

📦 You are saying:

“Give me a list of ages from 20 to 59 to test the model on.”

Part 2: .reshape(-1, 1)

- ML expects **2D shape**: like [[20], [21], [22], ...]
- -1 means: "I don't know how many rows, you figure it out"
- 1 = one column (age)

✅ **.reshape(-1, 1) — Means:**

- -1 🖐️ “I don’t know how many **rows** there will be. Python, you figure it out based on the total number of items.”
- 1 🖐️ “But I want **1 column**.”

📦 Final result is a column of ages like:

[[20],

[21],

[22],

...

[59]]

✅ **predicted_probs = model.predict_proba(ages)[: , 1]**

🧠 This line gets the **probability** of buying the product (1)

Let’s break it:

- model.predict_proba(ages) → Predicts the chance of **0 (not buy)** and **1 (buy)** for each age.
- [:, 1] → Means “give me the second column” → i.e., the **probability of buying**

📦 Example output:

Age 20 → 10% chance to buy

Age 30 → 40% chance to buy

Age 55 → 90% chance to buy

✓ `plt.figure(figsize=(10, 6))`

This sets the **canvas size** for the plot.

💡 It's like:

“Make my drawing area bigger (10 by 6 units) so it's easy to read.”

✓ `plt.scatter([age[0] for age in X], y, color='blue', label='Training Data')`

🎯 This draws the **actual data points** (training data).

- `age[0] for age in X` gets the actual age values from X, which is a 2D list.
 - Example: `[[22], [25]]` → we extract 22, 25
- `y` is the label (0 or 1)
- `color='blue'` means show dots in blue
- `label='Training Data'` → adds name in legend

📦 So this shows blue dots on graph for each person you trained on.

✓ `plt.plot(ages, predicted_probs, color='red', label='Prediction Curve')`

📊 This draws a **smooth line** showing the model's predictions.

- X-axis = Age
- Y-axis = Predicted probability of buying

💡 It's like saying:

“Hey, model, tell me how likely someone is to buy for each age, and draw that.”

✓ `plt.axvline(x=30, color='green', linestyle='--', label='Test Age: 30')`

This draws a **vertical line** at age 30 where we tested.

🟢 Green dashed line shows:

“Here's the person whose age is 30. Let's see where they land on the curve.”

✓ **Titles and Labels**

```
plt.title("Will a person buy based on age?")
```

```
plt.xlabel("Age")
```

```
plt.ylabel("Probability of Buying")
```

Adds the chart title and labels for axes:

-  X-axis = Age
 -  Y-axis = Probability (0 to 1)
-

plt.legend()

Adds the box in the corner that tells you:

- What the blue dots are (Training Data)
 - What the red line is (Model's prediction)
 - What the green line is (Test age)
-

plt.grid(True)

Adds little grid lines to make it easier to read the chart. Looks cool too 😎

plt.show()

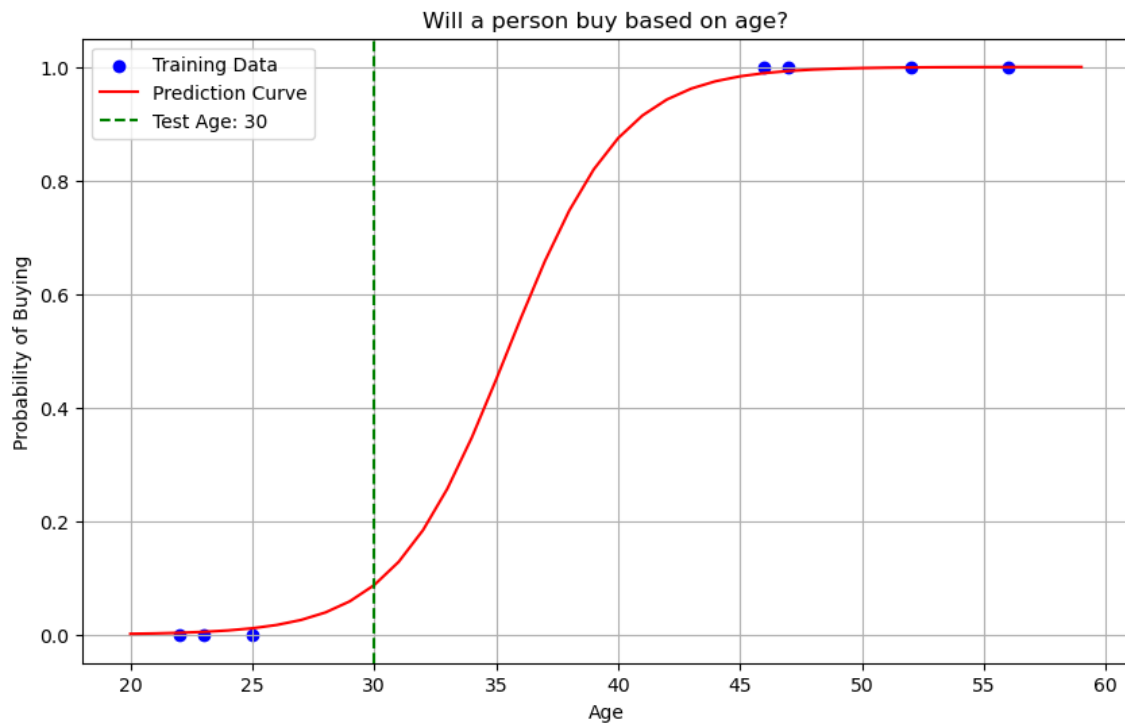
The final command to actually **display the graph**.

 Think of all the above commands as preparing the plot, and show() says:

“Alright, now show it on screen!”

What You'll See Visually

- Blue dots = real data
- Red curve = what model "learned"
- Green line = your test age



Let's build a **Mini Project** 🧪

🎯 **Project: Predict if a student will pass an exam based on how many hours they studied**

Dataset:

Hours Studied Passed (1 = Yes, 0 = No)

1	0
2	0
3	0
4	1
5	1
6	1
7	1

What you'll learn:

- Fit logistic regression to this data
 - Predict for new students (e.g., 2.5 hours)
 - Visualize the result using matplotlib
 - Understand decision boundary
-

Step-by-Step Plan:

1. Prepare the data
2. Train the model
3. Predict results
4. Visualize with Matplotlib
5. Understand decision boundary (the "cutoff" where model switches from 0 → 1)

```
from sklearn.linear_model import LogisticRegression
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
hours = np.array([1, 2, 3, 4, 5, 6, 7])
```

```
passed = np.array([0, 0, 0, 1, 1, 1, 1])
```

```
model = LogisticRegression()
```

```
model.fit(hours.reshape(-1,1),passed)
```

```
hours=[[14]]
```

```
prediction = model.predict(hours)
```

```
print("Prediction for hour 14:", prediction[0])
```

```
output: Prediction for hour 14: 1
```

```
# 1. Make a smooth line for hours from 0 to 8
```

```
hours_range = np.linspace(0, 12, 100).reshape(-1, 1)
```

```
probs_range = model.predict_proba(hours_range)[:, 1]
```

```
# 2. Plotting
```

```
plt.figure(figsize=(15, 5))
```

```
# Smooth prediction curve
```

```
plt.plot(hours_range, probs_range, color='green', linewidth=2, label='Predicted Probability')
```

```
# Red dots for new students
```

```
plt.scatter(new_students, predicted_probs, color='red', s=100, label='New Students')
```

```
# Labels and decorations
```

```
plt.xlabel('Hours Studied')
```

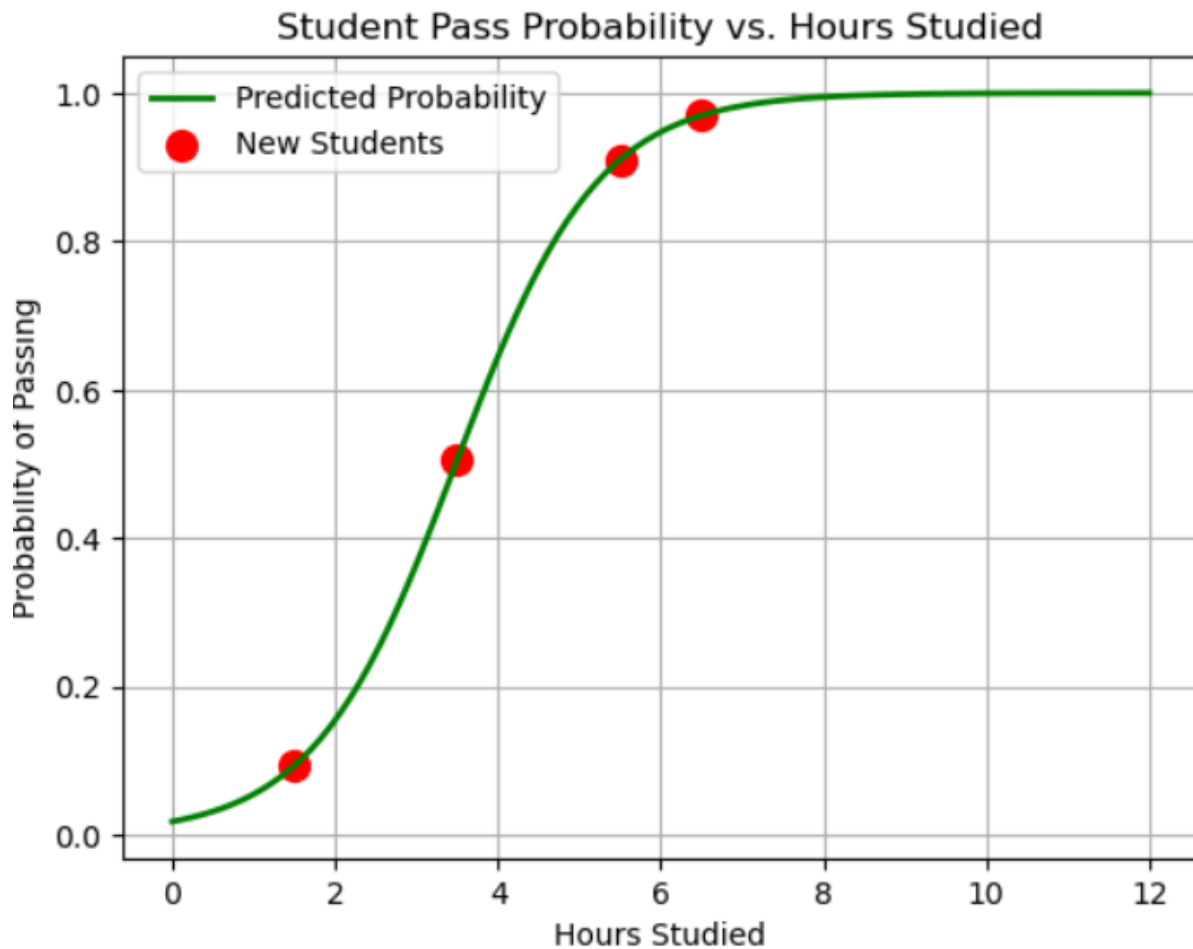
```
plt.ylabel('Probability of Passing')
```

```
plt.title('Student Pass Probability vs. Hours Studied')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```



🧠 Line-by-Line Breakdown

Line	What It Does
<code>np.linspace(0, 8, 100)</code>	Makes 100 points from 0 to 8 (smooth curve)
<code>.reshape(-1, 1)</code>	Reshapes to 2D input for model
<code>model.predict_proba(...)[..., 1]</code>	Gets only the probability of pass
<code>plt.figure(...)</code>	Makes the graph wider
<code>plt.plot(...)</code>	Draws the green prediction line
<code>plt.scatter(...)</code>	Draws big red dots for new students
<code>s=100</code>	Size of the red dots
<code>plt.xlabel(...)</code>	X-axis = hours studied
<code>plt.ylabel(...)</code>	Y-axis = probability of passing

Line**What It Does**`plt.title(...)`

Adds a title

`plt.legend()`

Adds label box

`plt.grid(True)`

Adds background grid

`plt.show()`

Shows the whole graph 🌟